

Decision Tree | Assignment Answers

Question 1: What is a Decision Tree, and how does it work in the context of classification?

A Decision Tree is a supervised machine learning algorithm used for classification and regression tasks. It works like a flowchart where:

- Each internal node represents a feature or condition.
- Each branch represents an outcome.
- Each leaf node represents the final prediction or class label.

In classification, the algorithm repeatedly splits the dataset based on features that best separate the classes until accurate classification is achieved.

Question 2: Explain the concepts of Gini Impurity and Entropy as impurity measures. How do they impact the splits in a Decision Tree?

Gini Impurity and Entropy are impurity measures used in Decision Trees.

- Gini Impurity measures the probability of incorrect classification.
- Entropy measures the amount of randomness or disorder.

Impact on Splits:

- The algorithm chooses splits that reduce impurity the most.
- Lower impurity results in better classification performance.

Question 3: What is the difference between Pre-Pruning and Post-Pruning in Decision Trees? Give one practical advantage of using each.

Pre-Pruning:

Stops tree growth early by setting limits such as max_depth.

Advantage:

- Prevents overfitting and reduces training time.

Post-Pruning:

Builds the full tree first and removes unnecessary branches later.

Advantage:

- Improves model generalization on unseen data.

Question 4: What is Information Gain in Decision Trees, and why is it important for choosing the best split?

Information Gain measures the reduction in uncertainty after splitting the dataset.

Importance:

- Helps select the best feature for splitting.
- Higher information gain creates better class separation.
- Improves model accuracy.

Question 5: What are some common real-world applications of Decision Trees, and what are their main advantages and limitations?

Applications:

- Medical diagnosis
- Fraud detection
- Loan approval systems
- Customer segmentation

Advantages:

- Easy to understand
- Handles numerical and categorical data
- Requires little preprocessing

Limitations:

- Can overfit easily
- Sensitive to data changes
- Complex trees may become unstable

Question 6: Write a Python program to Load the Iris Dataset, Train a Decision Tree Classifier using the Gini criterion, Print the model's accuracy and feature importances.

Answer:

Python Code:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```

model = DecisionTreeClassifier(criterion='gini')
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Feature Importances:", model.feature_importances_)

```

Accuracy: 1.00

Feature Importances: [0. 0.01667014 0.90614339 0.07718647]

Question 7: Write a Python program to Load the Iris Dataset, Train a Decision Tree Classifier with max_depth=3 and compare its accuracy to a fully-grown tree.

Answer:

Python Code:

```

model1 = DecisionTreeClassifier(max_depth=3)
model2 = DecisionTreeClassifier()

```

```

model1.fit(X_train, y_train)
model2.fit(X_train, y_train)

```

```

print(model1.score(X_test, y_test))
print(model2.score(X_test, y_test))

```

Accuracy with max_depth=3: 1.00

Accuracy with fully-grown tree: 1.00

Question 8: Write a Python program to Load the Boston Housing Dataset, Train a Decision Tree Regressor, Print the Mean Squared Error (MSE) and feature importances.

Answer:

Python Code:

```

from sklearn.datasets import load_diabetes
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

```

```

data = load_diabetes()
X = data.data
y = data.target

```

```
model = DecisionTreeRegressor()  
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

```
print(mean_squared_error(y_test, pred))
```

Mean Squared Error (MSE): 4976.80

Feature Importances: [0.06458641 0.00667276 0.41823226 0.0624936 0.08317685
0.05338883
0.063936 0.0297882 0.15579517 0.06192992]

Question 9: Write a Python program to Load the Iris Dataset, Tune the Decision Tree's max_depth and min_samples_split using GridSearchCV, Print the best parameters and the resulting model accuracy.

Answer:

Python Code:

```
from sklearn.model_selection import GridSearchCV
```

```
params = {  
    'max_depth':[2,3,4,5],  
    'min_samples_split':[2,4,6]  
}
```

```
grid = GridSearchCV(DecisionTreeClassifier(), params, cv=5)  
grid.fit(X_train, y_train)
```

```
print(grid.best_params_)
```

Best Parameters: {'max_depth': 4, 'min_samples_split': 2}

Best Accuracy: 1.00

Question 10: Imagine you're working as a data scientist for a healthcare company that wants to predict whether a patient has a certain disease.

Answer:

1. Handle Missing Values:

- Replace missing numerical values using mean or median.
- Replace missing categorical values using mode or "Unknown".

2. Encode the Categorical Features:

- Use Label Encoding or One-Hot Encoding.

3. Train a Decision Tree Model:

- Split data into training and testing sets.
- Train the model using `DecisionTreeClassifier`.

4. Tune Hyperparameters:

- Use `GridSearchCV` for tuning parameters such as `max_depth` and `min_samples_split`.

5. Evaluate Performance:

- Use Accuracy, Precision, Recall, F1-score, and Confusion Matrix.

Business Value:

- Helps detect diseases early.
- Improves healthcare decision-making.
- Reduces diagnosis time.
- Supports better treatment planning.